



Hacker Holidays

Welcome to The Byte Lotus

*Byte Lotus Wellness — Credenciales de Cognito Expuestas y
Volcado no Autorizado de DynamoDB*

Analista: Carlos Noboa

28 julio 2026



Tabla de contenido

Tabla de contenido	1
1. Introducción	3
2. Objetivo.....	3
3. Desarrollo del proceso	3
4. Conclusión	6
5. Bibliografía	6
6. Anexos	6

1. Introducción

Este informe documenta la resolución de un laboratorio de la serie Hacker Holidays de TryHackMe, ambientado en el hotel ficticio Byte Lotus, centrado en la aplicación de bienvenida «Byte Lotus Wellness». A diferencia de otros retos de la misma serie basados en fuerza bruta o en la manipulación de un agente conversacional, este laboratorio se sitúa en el ámbito de la seguridad en la nube y aborda una configuración incorrecta de Amazon Cognito: la aplicación, alojada como sitio estático en Amazon S3, entrega credenciales temporales de AWS a cualquier visitante sin exigir registro ni inicio de sesión, mediante un Identity Pool configurado para permitir acceso no autenticado.

El propio brief narrativo de la sala plantea que la huésped Lambo instaló la aplicación porque era gratuita, contaba con reseñas favorables escritas por la propia aplicación y no exigía cuenta alguna, y señala explícitamente que algo debe decidir qué datos mostrar sin que medie ningún inicio de sesión, sin comprobar con suficiente cuidado los límites de esa decisión. El objetivo declarado consiste en determinar el mecanismo que otorga esas credenciales y comprobar qué más es capaz de entregar más allá de los datos del propio usuario.

2. Objetivo

Identificar el mecanismo de AWS mediante el cual la aplicación Byte Lotus Wellness concede credenciales a un visitante sin autenticación previa, obtener credenciales válidas reproduciendo dicho mecanismo desde una máquina de ataque externa, determinar si el rol de identidad no autenticada asociado permite operaciones más amplias que las que ejecuta la propia aplicación, y extraer la flag contenida en el registro de otro huésped almacenado en la base de datos backend.

3. Desarrollo del proceso

La resolución del laboratorio siguió una metodología de reconocimiento e identificación de privilegios excesivos en la nube: análisis del frontend para localizar la configuración de AWS embebida en el cliente, reproducción manual del flujo de autenticación no autenticada de Cognito para obtener credenciales temporales, y verificación de los límites reales del rol IAM asociado frente a los límites que la propia aplicación respeta por convención. La tabla siguiente resume cronológicamente las acciones ejecutadas desde Kali Linux y el resultado observado en cada una.

N.º	Acción	Resultado observado
1	Análisis del brief y localización del objetivo	Se revisó el «Concierge Briefing» de la sala, que planteaba tres tareas: localizar el mecanismo de AWS que entrega credenciales a la aplicación, usarlas para volcar más datos que el propio registro, y recuperar la flag del registro de otro huésped. Se identificó como objetivo la URL http://complimentary-wellness-app-

N.º	Acción	Resultado observado
		332173347248.s3-website-us-east-1.amazonaws.com/, un sitio estático alojado en Amazon S3.
2	Reconocimiento de la aplicación en el navegador	Se accedió a la URL desde Firefox en Kali Linux. El sitio, bajo el nombre «Byte Lotus Wellness», mostró un panel de bienvenida sin pantalla de login, con el texto «No account needed — we set you up as a guest the moment you arrived» y un marcador de posición «Welcome! We don't have wellness data for you yet».
3	Inspección del DOM y de los scripts cargados	Mediante la pestaña Inspector de las herramientas de desarrollador se localizaron dos etiquetas <script> en el body: la referencia al SDK oficial de AWS para JavaScript (aws-sdk-2.1500.0.min.js) y un script propio de la aplicación (app.js), lo que orientó la búsqueda hacia este último como responsable de la lógica de acceso a AWS.
4	Lectura directa del código fuente de app.js	Se accedió a la ruta /app.js del mismo bucket S3, obteniendo el código en texto plano. En él se hallaron las constantes IDENTITY_POOL_ID (us-east-1:836c0949-292d-485b-b532-52d5ca7bb688), AWS_REGION (us-east-1) y TABLE_NAME (complimentary-GuestWellnessProfiles), junto con la lógica que solicita credenciales mediante AWS.CognitoIdentityCredentials y consulta DynamoDB con un getItem restringido a la clave guest_id generada y guardada en el localStorage del propio navegador.
5	Instalación de AWS CLI en la máquina de ataque	Al no estar disponible el binario aws en Kali, la instalación vía apt install awscli falló con errores 404 por un mirror de paquetes desactualizado. Se resolvió descargando e instalando manualmente el paquete oficial de AWS CLI v2 para arquitectura aarch64 desde awscli.amazonaws.com, sin depender del gestor de paquetes del sistema.
6	Obtención de un IdentityId no autenticado: aws cognito-identity get-id --identity-pool-id us-east-1:836c0949-292d-485b-b532-52d5ca7bb688 --region us-east-1	El comando, ejecutado sin ninguna credencial previa, devolvió el identificador us-east-1:4d571309-b07c-c6f2-d150-28ffcc7d699e, confirmando que el Identity Pool admite efectivamente el acceso no autenticado descrito en el brief.
7	Canje del IdentityId por credenciales temporales: aws cognito-identity get-credentials-for-identity --identity-id <IdentityId> --region us-east-1	La respuesta incluyó un juego de credenciales temporales (AccessKeyId, SecretKey, SessionToken) con expiración propia, equivalentes a las que recibe cualquier visitante en su propio navegador al cargar la aplicación.
8	Verificación del rol asumido: export de las credenciales como variables de entorno y	El comando confirmó la asunción del rol arn:aws:sts::332173347248:assumed-role/complimentary-cognito-unauth-role/CognitoIdentityCredentials, correspondiente al rol de invitado no autenticado vinculado al Identity Pool.

N.º	Acción	Resultado observado
	ejecución de aws sts get-caller-identity	
9	Volcado completo de la tabla DynamoDB: aws dynamodb scan --table-name complimentary-GuestWellnessProfiles --region us-east-1	A diferencia del getItem que ejecuta la aplicación (limitado al guest_id propio), el scan fue aceptado sin restricción por el rol asumido y devolvió los cinco registros completos de la tabla, cada uno con nombre, correo, teléfono, contraseña, ubicación y notas de un huésped distinto.
10	Localización de la flag	Entre los cinco registros devueltos, el correspondiente al guest_id guest-vip-042 contenía en su atributo notes el texto «If you're reading this, the wellness app's guest role can read every profile, not just its own», seguido de la flag THM{fr33_app_fr33_d4t4!}.

El hallazgo central del laboratorio combina dos decisiones que, por separado, podrían considerarse razonables, pero que juntas reproducen un patrón de privilegio excesivo. La primera es el propio diseño de la aplicación: activar el acceso no autenticado en el Identity Pool para evitar la fricción de un registro es una funcionalidad legítima de Amazon Cognito, pensada para escenarios en los que un visitante anónimo necesita credenciales mínimas antes de iniciar sesión. La segunda, y la que constituye la vulnerabilidad propiamente dicha, es que el rol IAM vinculado a esas identidades no autenticadas (complimentary-cognito-unauth-role) no estaba acotado a la operación que la aplicación realmente ejecuta: en lugar de restringirse a un dynamodb:GetItem sobre la clave del propio guest_id, el rol permitió sin restricción un dynamodb:Scan sobre la tabla completa.

Este contraste resulta especialmente relevante porque el código de app.js jamás invoca un scan; toda la lógica del cliente respeta el límite de leer únicamente el propio registro. Sin embargo, ese límite existe solo como convención en el código JavaScript que se ejecuta en el navegador, un entorno que el analista controla por completo y puede ignorar en cualquier momento sustituyendo la llamada por otra distinta usando las mismas credenciales válidas. La única frontera de seguridad real en una arquitectura de este tipo es la política IAM adjunta al rol que Cognito entrega, no la disciplina con la que el frontend decide usarlo; cuando esa política es más permisiva que lo que la interfaz necesita, cualquier visitante anónimo hereda automáticamente ese exceso de privilegio sin necesidad de vulnerar ningún control de acceso adicional.

El obstáculo encontrado al instalar AWS CLI mediante apt, motivado por un mirror de paquetes de Kali con referencias a versiones ya retiradas, no guarda relación con la seguridad de la aplicación objetivo y se resolvió instalando el paquete oficial directamente desde los servidores de Amazon, sin alterar la validez de los pasos posteriores. Por último, la flag no estaba oculta mediante ninguna técnica de ofuscación: se encontraba en texto plano dentro del atributo notes de un registro ajeno, redactada expresamente como advertencia dirigida a quien lograra

alcanzarla, lo que subraya que la severidad del hallazgo reside en el alcance de los datos expuestos —perfiles completos de huéspedes, incluidas contraseñas y coordenadas de ubicación— y no en la sofisticación de la técnica empleada para llegar a ellos.

4. Conclusión

El laboratorio reproduce, en un entorno controlado, un patrón de fallo habitual en arquitecturas serverless que delegan la obtención de credenciales de AWS directamente al navegador: la existencia de una identidad no autenticada resuelve un problema legítimo de experiencia de usuario, pero traslada la totalidad de la seguridad del sistema a la política IAM que se le asocia. Cuando dicha política concede permisos más amplios que los estrictamente necesarios para la operación prevista, cualquier visitante anónimo puede alcanzar datos que la interfaz nunca expone por diseño, sin necesidad de robar ninguna credencial ni de eludir mecanismo de autenticación alguno.

Este comportamiento corresponde a la categoría A01:2021 Broken Access Control del OWASP Top 10, en su variante de violación del principio de mínimo privilegio: el control de acceso existía únicamente como convención en el código cliente, mientras que el límite efectivo, definido en la política IAM, permitía una operación (`dynamodb:Scan`) muy superior a la que la aplicación necesitaba (`dynamodb:GetItem` sobre un único ítem). La mitigación recomendada por la propia documentación de Amazon Cognito para este escenario consiste en restringir el rol no autenticado a las acciones mínimas indispensables y, cuando resulte aplicable, condicionar el acceso a nivel de fila mediante la clave de contexto `cognito-identity.amazonaws.com:sub`, de forma que cada identidad solo pueda leer o escribir su propio registro incluso si en el futuro se introdujera por error una operación más amplia en el cliente.

5. Bibliografía

TryHackMe. (2026). Hacker Holidays: Byte Lotus Wellness [Room].

OWASP Foundation. (2021). OWASP Top 10:2021 – A01 Broken Access Control. https://owasp.org/Top10/2021/A01_2021-Broken_Access_Control/

Amazon Web Services. (n.d.). IAM roles – Amazon Cognito Developer Guide. <https://docs.aws.amazon.com/cognito/latest/developerguide/iam-roles.html>

Amazon Web Services. (n.d.). Security best practices for Amazon Cognito identity pools. <https://docs.aws.amazon.com/cognito/latest/developerguide/identity-pools-security-best-practices.html>

6. Anexos

Figura 1. Panel de bienvenida de Byte Lotus Wellness sin pantalla de login

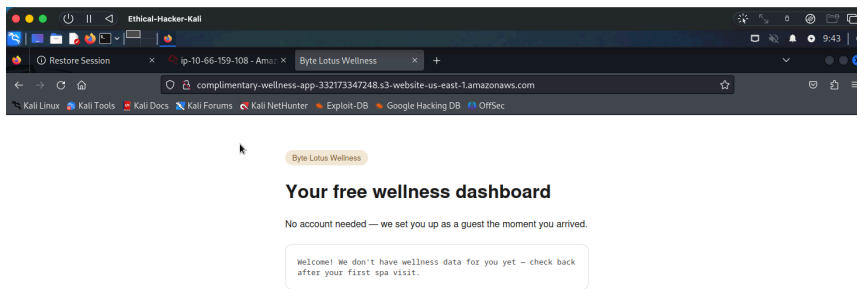


Figura 2. Identificación de app.js y del SDK de AWS en el Inspector

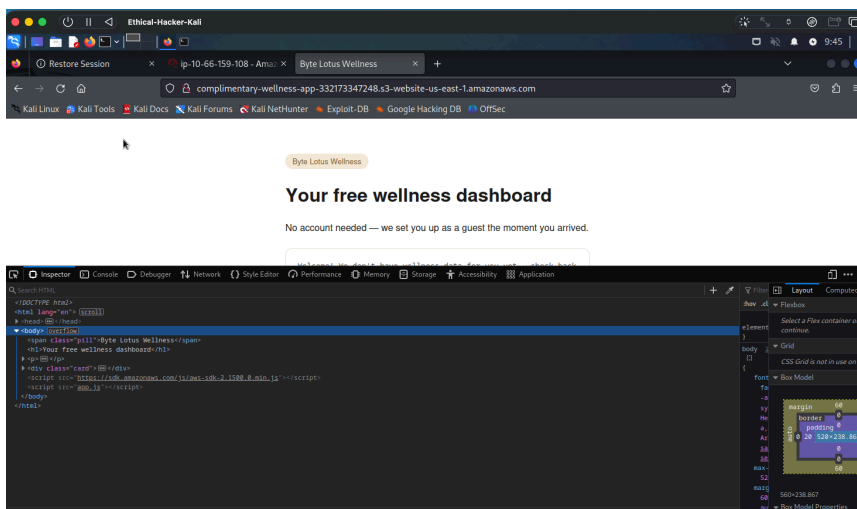


Figura 3. Código fuente de app.js con el Identity Pool ID y la tabla en texto plano

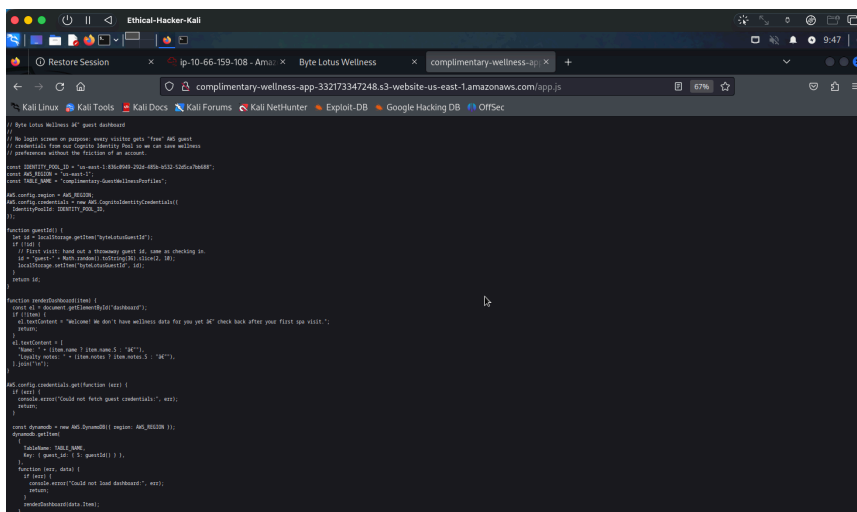


Figura 4. Obtención del IdentityId no autenticado con AWS CLI

```
(kali㉿kali)-[~]  
$ aws cognito-identity get-id \  
  --identity-pool-id us-east-1:836c0949-292d-485b-b532-52d5ca7bb688 \  
  --region us-east-1  
{  
  "IdentityId": "us-east-1:4d571309-b07c-c6f2-d150-28ffcc7d699e"  
}  
  
(kali㉿kali)-[~]  
$
```